

Review : What is an associative array ?

A collection of (key, value) pairs such that each possible key appears at most once in the collection.

Two major data structures used to implement an associative array :

search tree (set, map in C++ STL)
&

Hash Table

* Each Hash Table contains two parts :

1. Bucket Array (Table)

↓
size is fixed

use index to access each entry $A[0]$ $A[1]$..

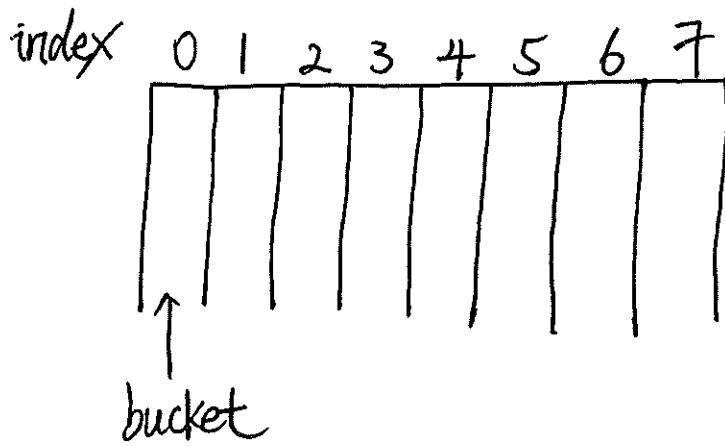
↓
Each entry of the Bucket Array is a Bucket to

store $\langle \text{key}, \text{value} \rangle$ pairs.

↑
multiple pairs can be put

in the same bucket

e.g.



bottom is open since we don't know how many pairs will be put in one bucket

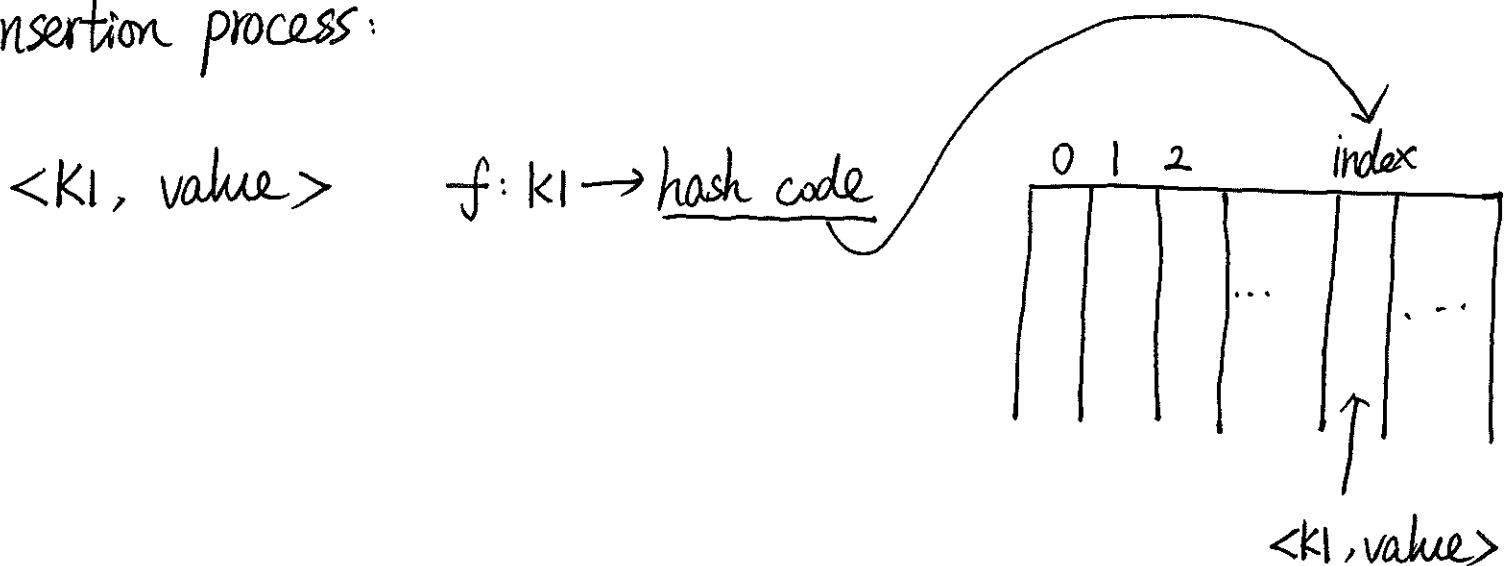
2. Hash function

Hash function maps key to a hash code (hash value)

$$f: \text{key} \mapsto \text{hash code}$$

The hash code is the index of a bucket array, and tells us which bucket we should use to store the $\langle \text{key}, \text{value} \rangle$

insertion process:



e.g. $f(\text{key}) = \text{key} \bmod 9$

What is the range of this hash function?

try some examples: $0 \bmod 9 = ?$

$$8 \bmod 9 = ?$$

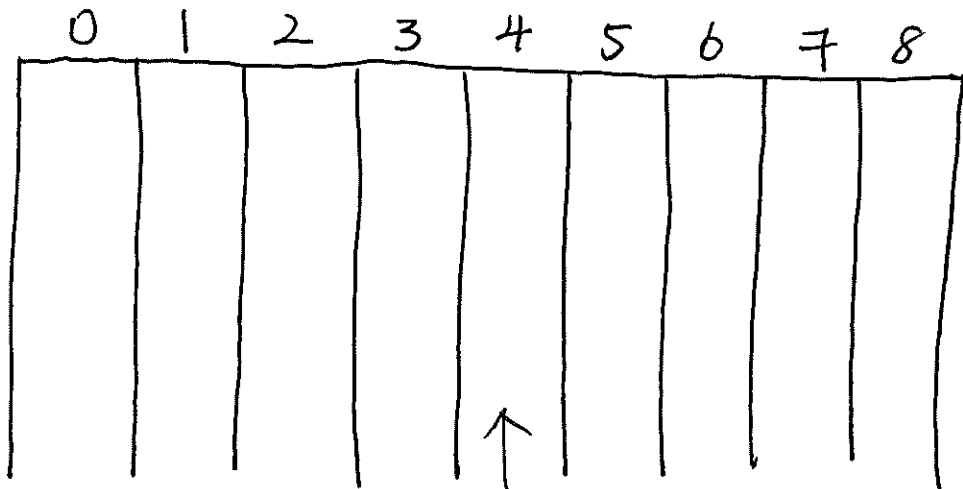
$$9 \bmod 9 = ?$$

$$21 \bmod 9 = ?$$

range: $\{0, 1, 2, 3, 4, 5, 6, 7, 8\}$

↳ all possible hash codes (hash values)

||
index: from 0 to 8

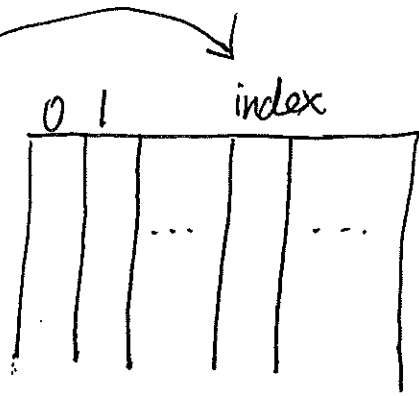


$\langle 13, \text{"Apple"} \rangle$ $f(13) = 13 \bmod 9 = 4$

$\langle 13, \text{"Apple"} \rangle$

search process:

key $f: \text{key} \mapsto \text{hash code}$



e.g. search (13)

$$f(13) = 4$$

* problem: Different keys may have the same hash value

e.g. $f(4) = 4 \bmod 9 = 4$

$\langle 4, \text{"Banana"} \rangle$

* This is called a hash collision.

Best case, there is no hash collision.

Each bucket has at most one pair.

Time complexity of search? $O(1)$

How to handle hash collision?

- A simple method: separate chaining

each bucket is a separate chain.

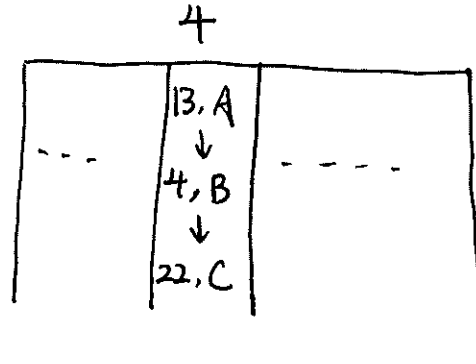
The size of a chain is not fixed and once a collision shows up, we simply add the element to the end of the chain.

eg.

<13, "Apple">

<4, "Banana">

<22, "Cherry">



Which data structure can we use to implement each bucket?

list

* A bucket array is an array (vector) of lists

In C++, `std::vector<std::list<int>>`

Implement a Hash Table

1. Data member

```
std::vector<std::list<int>> m_table;
```

2. Member functions

- Constructor

m_table is an empty vector.

We must create lists and push them in m_table.

How many lists (buckets)?

depends on the hash function.

e.g. $f(\text{key}) = \text{key} \bmod q$ How many buckets?

```
for(int i=0; i < # of lists; i++) {
```

```
    // create an empty new list
```

```
    // push it in m_table
```

```
}
```

list insert m_table[index].push_back
m_table[index].insert(...)

- insert(key, value)

1. $f(\text{key}) \Rightarrow \text{index}$ (hash code)

2. m_table[index] is the list we want.

3. go through the list m_table[index] (using list iterators)

if key exists, do nothing;

else, insert $\langle \text{key}, \text{value} \rangle$ into m_table[index]

- erase (key)

List
deletion

m_table[index].erase(pos)

1. $f(\text{key}) \Rightarrow \text{index}$

2. m_table[index] is what we want

3. go through the list

if key exists, delete the node from m_table[index]

else, do nothing.

- search (key)

1. $f(\text{key}) \Rightarrow \text{index}$

2. m_table[index] is the list

3. go through the list,

if find a key $k == \text{key}$ in the list, return K's value

else search failed.

e.g.

$f(\text{key}) = \text{key} \bmod 10$ (How many buckets?)

$\langle 3, A \rangle$	$\langle 25, D \rangle$	$\langle 17, G \rangle$
$\langle 5, B \rangle$	$\langle 35, E \rangle$	$\langle 33, H \rangle$
$\langle 12, C \rangle$	$\langle 9, F \rangle$	$\langle 36, I \rangle$

index	0	1	2	3	4	5	6	7	8	9
			$\langle 12, C \rangle$	$\langle 3, A \rangle$		$\langle 5, B \rangle$	$\langle 36, I \rangle$	$\langle 17, G \rangle$		$\langle 9, F \rangle$
				$\downarrow \uparrow$		$\downarrow \uparrow$				
				$\langle 33, H \rangle$		$\langle 25, D \rangle$				
						$\downarrow \uparrow$				
						$\langle 35, E \rangle$				

Time Complexity:

Best case:	insert	erase	search
	$O(1)$	$O(1)$	$O(1)$

Worst case:	insert	erase	search
	$O(n)$	$O(n)$	$O(n)$